

PD Tasks Team - Synthesis

Frequency Sweep Synthesis - tiny-NPU on ASAP7

Week 04 - Synthesis at 500 / 600 / 700 MHz with NPU Analysis

Design: tiny_npu - Neural Processing Unit / Transformer Inference Accelerator

PDK / Library: ASAP7 7nm predictive - asap7sc7p5t_AO RVT

Tool flow: OpenROAD-flow-scripts (Docker openroad/orfs) - Yosys 0.64 + slang - OpenROAD STA

Clock targets: 500 MHz (2.000 ns) - 600 MHz (1.667 ns) - 700 MHz (1.429 ns) on port clk

Github Repository: Fardeen2228/rv12-npu-synthesis-final

SUBMITTED TO

Asmicore

SUBMITTED BY

Fardeen

June 19, 2026

Table of Contents

1. Overview and Toolchain Note
2. RTL Exploration and Design Selection
 - 2.1 tiny-NPU source tree
 - 2.2 Identifying top module and clock port
3. Environment Setup and RTL Preparation
4. Clock Constraints and Design Configuration
 - 4.1 Per-frequency SDC files
 - 4.2 Final working config.mk files
5. Issues Faced and How Each Was Fixed
6. Day 1 - Running Synthesis at All Three Frequencies
7. Day 1 Metrics Table
8. Getting STA Numbers - make floorplan
9. Day 3 - Critical Path Analysis and Maximum Frequency
10. Day 2 - NPU Design-Specific Challenges and Questions
11. Frequency Scaling Observations
12. Deliverables and GitHub Submission Packaging
13. Key Facts Learned This Week
14. Reflection and Recommendation / Future Scope

1. Overview and Toolchain Note

This report documents the Week 4 frequency-sweep synthesis of the tiny-NPU design at 500 MHz, 600 MHz, and 700 MHz on the ASAP7 7nm predictive PDK. For each number reported, the command used and the raw tool-output slice are included in the relevant section. The work focuses on synthesis, structural metrics, post-synthesis floorplan STA, and NPU-specific design analysis.

Deviation from the assignment toolchain (OpenLane -> ORFS)

The assignment text is written around OpenLane config.json runs. In this setup, ASAP7 was already integrated inside OpenROAD-flow-scripts (ORFS), so the practical run was completed using the openroad/orfs Docker image. The core engines remain aligned with the assignment objective: Yosys synthesis, ABC technology mapping, and the ASAP7 standard-cell library. The main difference is the configuration format: ORFS uses config.mk and SDC files instead of OpenLane JSON CLOCK_PERIOD fields.

Item	Assignment expectation	Used here
Config format	config_500mhz.json, config_600mhz.json, config_700mhz.json	config_500.mk / config_600.mk / config_700.mk plus JSON wrappers for GitHub
Run command	python3 -m openlane --flow Classic --to synthesis config_*.json	make DESIGN_CONFIG=... synth
Clock constraint	CLOCK_PERIOD in JSON	create_clock in per-frequency SDC
Synthesis engine	Yosys + ABC speed-oriented mapping	Yosys 0.64 + slang frontend + ABC speed script
Cell library	asap7sc7p5t_AO	asap7sc7p5t_AO RVT NLDLM libraries
STA numbers	Usually in synthesis/STA report	make floorplan -> floorplan final timing report

2. RTL Exploration and Design Selection

The provided NPU repository was tiny-npu. The RTL directory contains bus, control, GEMM, memory, operation, package, and top-level files. The design is suitable for the Week 4 NPU assignment because it contains an NPU-style compute/control hierarchy: AXI bus logic, GEMM/MAC datapath, memory banks, and activation/normalization operation modules.

2.1 tiny-NPU source tree

Terminal 1 - RTL source files in tiny-npu.

```
find rtl -name "*.sv" | sort
rtl/bus/axi_dma_rd.sv
rtl/bus/axi_dma_wr.sv
rtl/bus/axi_lite_regs.sv
rtl/bus/axi_types.sv
rtl/ctrl/addr_gen.sv
rtl/ctrl/barrier.sv
rtl/ctrl/scoreboard.sv
rtl/ctrl/ucode_decode.sv
rtl/ctrl/ucode_fetch.sv
rtl/gemm/gemm_ctrl.sv
rtl/gemm/gemm_post.sv
rtl/gemm/mac_int8.sv
rtl/gemm/pe.sv
rtl/gemm/systolic_array.sv
rtl/mem/banked_sram.sv
rtl/mem/kv_cache_bank.sv
rtl/mem/sram_dp.sv
rtl/ops/exp_lut.sv
rtl/ops/gelu_engine.sv
rtl/ops/gelu_lut.sv
```

```
rtl/ops/layernorm_engine.sv
rtl/ops/mean_var_engine.sv
rtl/ops/recv_lut.sv
rtl/ops/reduce_max.sv
rtl/ops/reduce_sum.sv
rtl/ops/rmsnorm_engine.sv
rtl/ops/rope_engine.sv
rtl/ops/rsqrt_lut.sv
rtl/ops/silu_lut.sv
rtl/ops/softmax_engine.sv
rtl/ops/vec_engine.sv
rtl/pkg/fixed_pkg.sv
rtl/pkg/isa_pkg.sv
rtl/pkg/kv_pkg.sv
rtl/pkg/npu_pkg.sv
rtl/top.sv
```

2.2 Identifying the synthesis top and clock port

Terminal 2 - top.sv confirms clock port clk and reset rst_n.

```
grep -n "input.*clk\\|input.*rst\\|input.*reset" rtl/top.sv | head -30
24:   input wire                               clk,
25:   input wire                               rst_n,
```

Final synthesis selection: DESIGN_NAME = top, CLOCK_PORT = clk, active-low reset = rst_n. The top-level file is rtl/top.sv.

3. Environment Setup and RTL Preparation

OpenROAD-flow-scripts was already installed on the machine and the ASAP7 platform was present at flow/platforms/asap7. The NPU RTL was copied into the standard ORFS source tree under flow/designs/src/tiny_npu, while the design-specific configuration and SDC files were placed under flow/designs/asap7/tiny_npu.

Terminal 3 - ORFS design tree and RTL copy operation.

```
cd $HOME/OpenROAD-flow-scripts/flow
mkdir -p designs/asap7/tiny_npu
mkdir -p designs/src/tiny_npu
rm -rf designs/src/tiny_npu/*
cp -r $HOME/asmicore_week4/tiny-npu/rtl/* designs/src/tiny_npu/

find designs/src/tiny_npu -name "*.sv" | wc -l
36
```

4. Clock Constraints and Design Configuration

4.1 Per-frequency SDC files

ASAP7 Liberty timing in this ORFS setup is in picoseconds, so the assignment clock periods were written as 2000, 1667, and 1429 in the SDC files. These correspond to 2.000 ns, 1.667 ns, and 1.429 ns.

Terminal 4 - SDC constraints for 500, 600, and 700 MHz.

```
cat > designs/asap7/tiny_npu/constraint_500.sdc << 'EOF'
create_clock -name clk -period 2000 [get_ports clk]
EOF

cat > designs/asap7/tiny_npu/constraint_600.sdc << 'EOF'
create_clock -name clk -period 1667 [get_ports clk]
EOF
```

```
cat > designs/asap7/tiny_npu/constraint_700.sdc << 'EOF'
create_clock -name clk -period 1429 [get_ports clk]
EOF
```

4.2 Final working config.mk files

Terminal 5 - final ORFS config.mk generation loop.

```
for FREQ in 500 600 700; do
cat > designs/asap7/tiny_npu/config_${FREQ}.mk << EOF
export PLATFORM = asap7
export DESIGN_NICKNAME = tiny_npu_${FREQ}
export DESIGN_NAME = top
export VERILOG_INCLUDE_DIRS = ./designs/src/tiny_npu/pkg
export VERILOG_FILES =      ./designs/src/tiny_npu/pkg/fixed_pkg.sv
./designs/src/tiny_npu/pkg/isa_pkg.sv    ./designs/src/tiny_npu/pkg/kv_pkg.sv
./designs/src/tiny_npu/pkg/npk_pkg.sv    ./designs/src/tiny_npu/bus/axi_types.sv  \$(filter-out
./designs/src/tiny_npu/bus/axi_types.sv,\$(sort \$(wildcard ./designs/src/tiny_npu/bus/*.sv)))  \$(
(sort \$(wildcard ./designs/src/tiny_npu/ctrl/*.sv))  \$(sort \$(wildcard
./designs/src/tiny_npu/gemm/*.sv))  \$(sort \$(wildcard ./designs/src/tiny_npu/mem/*.sv))  \$(sort
\$(wildcard ./designs/src/tiny_npu/ops/*.sv))  ./designs/src/tiny_npu/top.sv
export SDC_FILE = ./designs/asap7/tiny_npu/constraint_${FREQ}.sdc
export SYNTH_HDL_FRONTEND = slang
export SLANG_PLUGIN_PATH = /OpenROAD-flow-scripts/tools/install/yosys/share/yosys/plugins/slang.so
export MAX_FANOUT = 8
export DIE_AREA = 0 0 1000 1000
export CORE_AREA = 10 10 990 990
export PLACE_DENSITY = 0.60
export TNS_END_PERCENT = 100
EOF
done
```

5. Issues Faced and How Each Was Fixed

#	Issue / problem	Root cause	Fix that worked
1	OpenLane assignment format vs ORFS environment	Assignment uses OpenLane JSON, but ASAP7 was ready inside ORFS on this machine	Used ORFS for actual runs and prepared OpenLane-style JSON wrappers for GitHub submission
2	SystemVerilog packages/imports	The NPU RTL uses SystemVerilog packages such as npu_pkg and axi_types	Enabled SYNTH_HDL_FRONTEND = slang and set SLANG_PLUGIN_PATH
3	Package ordering	Packages must be read before modules that import them	Listed pkg files first and axi_types.sv before other bus RTL
4	STA numbers not printed by synth alone	ORFS synth target runs Yosys/ABC and writes synth_stat.txt, but full STA numbers are available after floorplan	Ran make floorplan and extracted WNS/TNS/worst slack from floorplan reports
5	Unconstrained IO warnings during floorplan	Only the main clock was constrained; IO delays were not defined for this synthesis-focused task	Documented warnings; core clock timing still met with no setup violations

6. Day 1 - Running Synthesis at All Three Frequencies

The three synthesis runs were performed separately by changing only the frequency-specific config file. The 500 MHz run is shown as a representative clean run; 600 MHz and 700 MHz completed in the same way.

Terminal 6 - canonicalization build check for 500 MHz.

```
make DESIGN_CONFIG=./designs/asap7/tiny_npu/config_500.mk clean_synth
make DESIGN_CONFIG=./designs/asap7/tiny_npu/config_500.mk synth 2>&1 | tee
/tmp/tiny_npu_synth_500.log
```

```
Top level design units:
top
```

```
Build succeeded: 0 errors, 0 warnings
Elapsed time: 0:00.96[h:]min:sec. CPU time: user 0.95 sys 0.05 (104%). Peak memory: 104472KB.
```

Terminal 7 - three synthesis result trees were generated.

```
make DESIGN_CONFIG=./designs/asap7/tiny_npu/config_600.mk clean_synth
make DESIGN_CONFIG=./designs/asap7/tiny_npu/config_600.mk synth 2>&1 | tee
/tmp/tiny_npu_synth_600.log

make DESIGN_CONFIG=./designs/asap7/tiny_npu/config_700.mk clean_synth
make DESIGN_CONFIG=./designs/asap7/tiny_npu/config_700.mk synth 2>&1 | tee
/tmp/tiny_npu_synth_700.log

ls results/asap7 | grep tiny_npu
tiny_npu_500
tiny_npu_600
tiny_npu_700
```

7. Day 1 Metrics Table

The structural numbers below were extracted from reports/asap7/tiny_npu_<FREQ>/base/synth_stat.txt. The timing values were extracted after floorplan from floorplan final reports.

Metric	500 MHz	600 MHz	700 MHz	500 -> 700 change	Timing met all 3?
Clock period	2.000 ns	1.667 ns	1.429 ns	-0.571 ns	Yes
Cell count	3,482	3,482	3,482	0%	Yes
Chip area	537.345900 um^2	537.345900 um^2	537.345900 um^2	0%	Yes
Flip-flop count	714	714	714	0%	Yes
WNS	0.00 ns	0.00 ns	0.00 ns	No negative slack	Yes
TNS	0.00 ns	0.00 ns	0.00 ns	No timing debt	Yes
Positive worst slack	+1.471 ns	+1.138 ns	+0.900 ns	Margin reduced by 0.571 ns	Yes
Most common cell	DFFASRHQNx1_ASA P7_75t_R = 714	Same	Same	0%	Yes

Terminal 8 - cell count, area, and sequential area extraction.

```
grep -E "cells|Chip area|Design area|sequential" reports/asap7/tiny_npu_${FREQ}/base/synth_stat.txt

500 MHz:
3482 cells
Chip area for module '      op': 537.345900
of which used for sequential elements: 270.663120 (50.37%)

600 MHz:
3482 cells
Chip area for module '      op': 537.345900
of which used for sequential elements: 270.663120 (50.37%)

700 MHz:
3482 cells
Chip area for module '      op': 537.345900
of which used for sequential elements: 270.663120 (50.37%)
```

Terminal 9 - flip-flop count and sequential element evidence.

```
grep -iE "DFF|SDF|seq|flop" reports/asap7/tiny_npu_${FREQ}/base/synth_stat.txt | head -20
```

```
714 270.663 714 270.663 DFFASRHQNx1_ASAP7_75t_R
of which used for sequential elements: 270.663120 (50.37%)
```

Terminal 10 - top cell types at 700 MHz.

```
grep "ASAP7" reports/asap7/tiny_npu_700/base/synth_stat.txt | sort -k1,1nr | head -10
714 DFFASRHQNx1_ASAP7_75t_R
498 NAND2x1_ASAP7_75t_R
466 OA21x2_ASAP7_75t_R
401 BUFx2_ASAP7_75t_R
195 INVx1_ASAP7_75t_R
180 OA22x2_ASAP7_75t_R
139 AO21x1_ASAP7_75t_R
129 NOR2x1_ASAP7_75t_R
126 XNOR2x2_ASAP7_75t_R
107 AND2x2_ASAP7_75t_R
```

8. Getting STA Numbers - make floorplan

The ORFS synth target created the mapped netlist and synth_stat.txt but did not provide the final floorplan STA table. Therefore, make floorplan was run for each frequency. This generated floorplan timing reports and confirmed that there were no setup violations.

Terminal 11 - floorplan run and setup repair summary.

```
for FREQ in 500 600 700; do
  make DESIGN_CONFIG=./designs/asap7/tiny_npu/config_${FREQ}.mk floorplan 2>&1 | tee
  /tmp/tiny_npu_floorplan_${FREQ}.log
done

[INFO RSZ-0098] No setup violations found
floorplan final report_design_area
Design area 578 um^2 0% utilization.
```

Terminal 12 - WNS, TNS, and positive worst slack extraction.

```
grep -iE "wns|tns|worst slack|slack" reports/asap7/tiny_npu_${FREQ}/base/*floorplan*.rpt | tail -30

500 MHz:
tns max 0.00
wns max 0.00
worst slack max 1471.01 ps

600 MHz:
tns max 0.00
wns max 0.00
worst slack max 1138.01 ps

700 MHz:
tns max 0.00
wns max 0.00
worst slack max 900.01 ps
```

Frequency	TNS max	WNS max	Worst slack max	Interpretation
500 MHz	0.00	0.00	1471.01 ps = +1.471 ns	Timing met with large margin
600 MHz	0.00	0.00	1138.01 ps = +1.138 ns	Timing met
700 MHz	0.00	0.00	900.01 ps = +0.900 ns	Timing met, smallest margin

9. Day 3 - Critical Path Analysis and Maximum Frequency

9.1 Reading the worst setup path at 700 MHz

Terminal 13 - 700 MHz worst clocked path.

```
grep -iE "Startpoint|Endpoint|slack|Path Group|data arrival|data required"
reports/asap7/tiny_npu_700/base/*floorplan*.rpt | head -80
```

```
Startpoint: u_dma_rd.m_axi_araddr[1]$_DFFE_PN0P_
Endpoint: u_dma_rd.m_axi_arlen[7]$_DFFE_PN0P_
Path Group: clk
data arrival time 514.24
data required time 1414.25
slack (MET) 900.01
```

Property	500 MHz	600 MHz	700 MHz
Worst setup startpoint	u_dma_rd.m_axi_araddr[1]	u_dma_rd.m_axi_araddr[1]	u_dma_rd.m_axi_araddr[1]
Worst setup endpoint	u_dma_rd.m_axi_arlen[7]	u_dma_rd.m_axi_arlen[7]	u_dma_rd.m_axi_arlen[7]
Path group	clk	clk	clk
Data arrival time	514.24 ps	514.24 ps	514.24 ps
Worst setup slack	1471.01 ps	1138.01 ps	900.01 ps
Timing status	MET	MET	MET

9.2 Maximum frequency estimate

At 700 MHz, the worst clocked path arrival time is 514.24 ps, while the required time is 1414.25 ps. Since the 700 MHz run still has +900.01 ps slack, the design clearly meets timing at all three assignment frequencies. A simple idealized estimate from this observed data path is $f_{max} > 700$ MHz; using the 514.24 ps arrival alone gives a theoretical limit around 1.94 GHz, but this is optimistic because IO constraints, routing parasitics, placement, and signoff derating are not fully included at this early floorplan stage.

10. Day 2 - NPU Design-Specific Challenges and Questions

10.1 Challenge 1 - activation-related logic in the netlist

Terminal 14 - activation-related names from the flattened netlist.

```
grep -iE "relu|sigmoid|tanh|gelu|silu|lut|softmax|exp|norm"
results/asap7/tiny_npu_700/base/1_2_yosys.v | head -50
```

```
DFFASRHQNx1_ASAP7_75t_R gelu_busy_r...
DFFASRHQNx1_ASAP7_75t_R gelu_done...
DFFASRHQNx1_ASAP7_75t_R gelu_timer[0..15]...
DFFASRHQNx1_ASAP7_75t_R layernorm_timer[0..15]...
DFFASRHQNx1_ASAP7_75t_R softmax_timer[0..15]...
```

The final flattened netlist contains activation-related names such as gelu, layernorm, and softmax, but the visible instances are mostly status/timer registers rather than a separate unflattened sigmoid block. This is expected because Yosys flattened the hierarchy and optimized logic during mapping.

10.2 Challenge 2 - datapath/control dominance

The final netlist has 3,482 cells. The most common cell is a DFF, with 714 instances. The critical path at 700 MHz is not in the MAC or activation block; it is in DMA read address/control logic. This indicates that, for this tiny-NPU configuration, control and data-movement logic is more visible in the timing bottleneck than the arithmetic datapath.

10.3 Challenge 3 - INT8 multiplier / MAC evidence

The RTL source tree includes `gemm/mac_int8.sv`, `pe.sv`, and `systolic_array.sv`, confirming that the design contains INT8 MAC/GEMM-oriented RTL. However, the gate-level synthesis result is flattened, so a clean per-MAC unit cell count cannot be directly read from the final global `synth_stat.txt`. The mapped netlist top-cell distribution shows arithmetic-related cells such as XNOR2, XOR2, half-adders, NANDs, and AO/OA gates, but not a retained per-module MAC statistic.

10.4 Questions Q1-Q4

Q#	Question	Answer
Q1	Does sigmoid appear as a distinct block? How many cells compared to one MAC unit?	Sigmoid does not appear as a distinct preserved block in the final flattened netlist. Activation-related names are mostly <code>gelu</code> , <code>softmax</code> , and <code>layernorm</code> control/timer registers. Because the hierarchy was flattened, a direct cell-count comparison between sigmoid and one MAC unit could not be extracted from <code>synth_stat.txt</code> .
Q2	At 700 MHz, is the critical path inside a MAC multiplier, activation function, or control logic?	The worst clocked path at 700 MHz is from <code>u_dma_rd.m_axi_araddr[1]</code> to <code>u_dma_rd.m_axi_arlen[7]</code> , with +900.01 ps slack. Therefore, the observed critical path is in DMA/read-control logic, not in a MAC multiplier or activation function.
Q3	If MAC units are doubled, how would cell count and WNS change?	Cell count and area would increase approximately linearly for the replicated MAC datapath. Maximum frequency would not automatically improve because it depends on the longest path. WNS could worsen if duplicated MACs increase fanout, routing pressure, or control complexity.
Q4	Compare NPU with TPU. What architectural difference affects frequency scaling?	A TPU systolic array is highly regular, while an NPU also includes activation/normalization, softmax, DMA, register control, and sequencing logic. In this run, the timing bottleneck appeared in data movement/control rather than pure MAC compute, showing that NPU frequency behavior is influenced by non-compute support logic.

11. Frequency Scaling Observations

Cell count 500 -> 700 MHz: 0% change. The same mapped netlist structure was generated at all three frequencies. Area also remained constant at 537.345900 μm^2 . This behavior is expected when the design already meets all requested timing constraints with a comfortable margin and the speed script does not need to restructure the netlist.

Timing margin reduced as frequency increased. The positive worst slack moved from +1.471 ns at 500 MHz to +1.138 ns at 600 MHz and +0.900 ns at 700 MHz. Therefore, the design still meets timing at all three targets, but the remaining margin is smallest at 700 MHz.

Most common cell type: `DFFASRHQNx1_ASAP7_75t_R`, with 714 instances. Sequential cells occupy 270.663120 μm^2 , which is 50.37% of the reported cell area. This means registers are a major structural component in the mapped tiny-NPU design.

12. Deliverables and GitHub Submission Packaging

The GitHub update should include the report, frequency analysis markdown, config wrappers, ORFS config/SDC files, and per-frequency synthesis folders. Since the actual ORFS result files live on the local machine, the commands below copy those real files into the existing `rv12-npu-synthesis-final` repository.

Terminal 15 - GitHub repository update command pattern.

```
# Run from normal Ubuntu terminal, not inside the ORFS container
REPO=$HOME/rv12-npu-synthesis-final
FLOW=$HOME/OpenROAD-flow-scripts/flow
mkdir -p $REPO/npu/flow_config
mkdir -p $REPO/npu/synthesis_500 $REPO/npu/synthesis_600 $REPO/npu/synthesis_700

cp $FLOW/designs/asap7/tiny_npu/config_*.mk $REPO/npu/flow_config/
cp $FLOW/designs/asap7/tiny_npu/constraint_*.sdc $REPO/npu/flow_config/

for FREQ in 500 600 700; do
  cp $FLOW/results/asap7/tiny_npu_${FREQ}/base/1_2_yosys.v $REPO/npu/synthesis_${FREQ}/
  cp $FLOW/logs/asap7/tiny_npu_${FREQ}/base/1_2_yosys.log $REPO/npu/synthesis_${FREQ}/
  cp $FLOW/reports/asap7/tiny_npu_${FREQ}/base/synth_stat.txt $REPO/npu/synthesis_${FREQ}/
  cp $FLOW/reports/asap7/tiny_npu_${FREQ}/base/2_floorplan_final.rpt $REPO/npu/synthesis_${FREQ}/
2>/dev/null || true
  cp $FLOW/results/asap7/tiny_npu_${FREQ}/base/2_floorplan.sdc $REPO/npu/synthesis_${FREQ}/
2>/dev/null || true
done

git -C $REPO add npu
git -C $REPO commit -m "Add tiny NPU ASAP7 synthesis results"
git -C $REPO push
```

Deliverable	Status	Location
config_500/600/700mhz.json	Prepared as wrappers	npu/synthesis_500/ etc.
ORFS config.mk and SDC	Prepared	npu/flow_config/
synthesis_500/600/700 directories	To be copied from ORFS results	npu/synthesis_500, npu/synthesis_600, npu/synthesis_700
Day 1 metrics table	Completed	Section 7
Day 2 NPU questions Q1-Q4	Completed	Section 10.4
Critical path analysis	Completed	Section 9
Frequency analysis markdown	Prepared separately	npu/frequency_analysis.md

13. Key Facts Learned This Week

- ASAP7 frequency sweeps need careful clock-period units; in this ORFS setup SDC periods were written in picoseconds.
- SystemVerilog package-heavy designs are easier to synthesize in ORFS by enabling the slang frontend.
- Synthesis area and cell count came from synth_stat.txt, while WNS/TNS came from floorplan timing reports.
- For tiny-NPU, cell count and area stayed constant across 500, 600, and 700 MHz because the same netlist met all constraints.
- The worst observed 700 MHz clocked path was in DMA read control, not inside a MAC or activation unit.

14. Reflection and Recommendation / Future Scope

The tiny-NPU synthesis run was clean and timing-safe for all three assignment frequencies. The most important result is that the timing bottleneck did not appear in the expected MAC or activation logic; instead, the worst clocked path was in DMA address/control logic. This is an important architectural lesson: in accelerators, the compute datapath is not always the limiting factor. Data movement, control, and memory-interface logic can become equally important for high-frequency implementation.